

Serviços e Interfaces para a Cloud - SIC_TSIW

UNIDADE CURRICULAR

REST application definition and API documentation

FICHA

Group work: This exercise should be completed in groups.

Deadline: Submit on Moodle until 23h:59 of 20/10/2025

In this exercise you'll put into practice your knowledge about designing REST APIs with best practices and API documentation.

Use the microservices application from the previous exercise, in which you divided a monolithic application into multiple services (if you haven't done the previous exercise you can use the solution code from here: <https://github.com/leandrocotsa/sic-monolith-to-microservices-exercise/tree/pl-02-solution-of-split>), and create the missing CRUD operations to each service following REST practices. The goal is for you to demonstrate your understanding of **REST API design**, **endpoint structuring**, proper use of **HTTP methods**, **status codes** and **API documentation**.

You should focus on:

- Designing clear and consistent URIs (there are some missing operations)
- Using the appropriate HTTP methods (GET, POST, PUT, DELETE)
- Returning appropriate status codes and messages (200, 201, 400, 404, 409, etc.)
- Generating documentation for API consumers using Swagger

You should submit your **code** and the **swagger files** containing all the endpoints, response bodies and status codes.

Technical hints:

- In the user service, add an email field to the users. When creating a user, we should not allow users with the same email.
- For this exercise, it is not required to set up a database. You can store and manage data using local objects in your application.

Generating Swagger API specs

To automatically generate Swagger API specs from your endpoints you need to install the following modules in each of your services:

```
npm install swagger-autogen swagger-ui-express
```

Then you should create a swagger.js file:

```
const swaggerAutogen = require("swagger-autogen")();
const doc = {
  info: {
    title: "My API",
    description: "Swagger documentation",
  },
  host: "localhost:3003",
  schemes: ["http"],
  tags: [ // the sections that will be presented in swagger page
    { name: "Users", description: "Users related endpoints" },
  ],
  definitions: { // the objects used in the request and response bodies
    GetUser: { // GET response bodies come with id
      id: 123,
      name: "Example Name",
      email: "example@mail.com"
    },
    CreateUser: { // POST/PUT request bodies are sent without id
      name: "Example Name",
      email: "example@mail.com"
    }
  }
};

const outputFile = "./swagger-output.json";
const endpointsFiles = ["./*.js"];

swaggerAutogen(outputFile, endpointsFiles, doc);
```

You also need to add these to your main application file app.js:

```
const swaggerUi = require("swagger-ui-express");
const swaggerFile = require("./swagger-output.json");

app.use("/api-docs", swaggerUi.serve, swaggerUi.setup(swaggerFile));
```

For your endpoints to show up in the Swagger page you need to add some annotations to your endpoints, as follows:

```
app.post("/users", (req, res) => {  
  /*  
    #swagger.tags = ['Users']  
    #swagger.parameters['body'] = {  
      in: 'body',  
      description: 'New user object',  
      required: true,  
      schema: { $ref: '#/definitions/CreateUser' }  
    }  
    #swagger.responses[201] = { description: 'User created successfully', schema: {  
$ref: '#/definitions/GetUser'} }  
    #swagger.responses[409] = { description: 'Email already exists' }  
  */  
}
```

The "tags" and "refs" used above should match whatever you defined in your swagger.js file.

Once everything above is done, you can generate the Swagger API spec by running:

```
node swagger.js
```

After generating the swagger file, start the application and if you access <http://localhost:<your-service-port>/api-docs> you can see the API specifications of the service.